Welcome!

COMP 3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 2: REVISITING OBJECT ORIENTED PROGRAMMING

Agenda

1. Datetime
2. Essential Object Oriented Programming.
   (All about Classes & Objects)

DATETIME

An interesting idea

Running programs while you're at your computer is fine

But it's also useful for programs to execute without our direct supervisions

We can use our computer's clock to run code:

- At some specified time and date
- At regular intervals

We can also write programs that launch other programs on a schedule

Let’s start with the time module

Your system’s clock is set to a specific date, time, and time zone

The **time** module gives us access to the current time

We will look at two functions:

1. time.time( )
2. time.sleep( )

Everything is predicated on the Unix epoch

https://docs.python.org/3.7/library/time.html

Unix epoch

Time reference used in programming

The unix epoch is the *“point where time starts”*

Everything begins on January 01 1970 at 00:00:00, Coordinated Universal Time (UTC)

We count the number of seconds since this time and call it **seconds since the epoch**

time.time( )

```
import time
time.time( ) #1705341919.1878183
```

It returns the number of seconds that have elapsed “since time began”

It’s a float (very precise!)

We can use this to profile* code!

* Profile: measure how long a piece of code takes to run

Example

```
import time


product = 1
start_time = time.time()
for i in range(1, 100000): #perform operation that takes a lot of time
    product = product * i
end_time = time.time()
print('The result is %s digits long.' % (len(str(product))))
print('Took %s seconds to calculate.' % (end_time - start_time))
```

time.sleep(seconds)

We can pause our code for a specified number of seconds (can be a float for precision)

The time.sleep(seconds) function will *block* until at least the specified number of seconds has elapsed

* *Block*: will not return and release your program to execute other code

Example

```
import time
for _ in range(3):
    print(‘Tick’)
    time.sleep(1)
    print(‘Tock’)
    time.sleep(1)
```

The datetime module

The time module is good for getting a Unix epoch timestamp to work with

What if we want a date in a more convenient format?

Use the ***datetime module***

Lets us manipulate dates and times

The datetime module contains a new datatype called datetime

a datetime *represents a specific moment in time*

datetime object

The datetime data type in the datetime module can be used to create datetime objects

A datetime object uses integers to store:

1. Year
2. Month
3. Day
4. Hour
5. Minute
6. Second

Converting time types

We can convert a Unix epoch timestamp to a datetime object using the datetime module's **fromtimestamp**( ) function

The date and time of the datetime will be converted to the local time zone

```
import datetime

print(datetime.datetime.fromtimestamp(1000000)) #1970-01-12 05:46:40

import time

print(datetime.datetime.fromtimestamp(time.time())) #2026-01-13 16:25:04.843986 Whatever the time is at the moment you ran the code
```

Comparing times

datetime objects can be compared using the comparison operators

The later datetime object is the “greater” value

```
import datetime

halloween = datetime.datetime(2018, 10, 31)
nyd = datetime.datetime(2019, 1, 1)
oct_31 = datetime.datetime(2018, 10, 31)
print(halloween == oct_31) #True
print(halloween > nyd) #False
print (nyd != oct_31) #True
```

Comparing times

HANDY TIP:

Change “import datetime” to “from datetime import datetime”

This imports just the **datetime class** from the **datetime module**.

This allows you to use the **datetime class** by calling “**datetime**” vs “**datetime.datetime**”

```
from datetime import datetime

halloween = datetime(2018, 10, 31)
nyd = datetime(2019, 1, 1)
oct_31 = datetime(2018, 10, 31)
print(halloween == oct_31) #True
print(halloween > nyd) #False
print (nyd != oct_31) #True
```

Comparing times

ULTRA HANDY TIP:

Change "import datetime" to "from datetime import *"

This imports **ALL CLASSES** from the **datetime module**.

This allows you to use **any class** within the module by their class name class by calling "**classname**" vs "**datetime.classname**"

Note – PEP 8 considers import * bad practice as it pollutes the namespace

```
from datetime import *

halloween = datetime(2018, 10, 31)
nyd = datetime(2019, 1, 1)
oct_31 = datetime(2018, 10, 31)
print(halloween == oct_31) #True
print(halloween > nyd) #False
print (nyd != oct_31) #True
```

The timedelta datatype

datetime represents a *point in time*

timedelta represents a ***duration of time***

We can create a timedelta that stores:

1. Weeks
2. Days
3. Hours
4. Minutes
5. Seconds
6. Milliseconds
7. Microseconds

timedelta example

```
import datetime

#instantiate timedelta by subtracting datetimes

halloween = datetime.datetime(2018, 10, 31)
nyd = datetime.datetime(2019, 1, 1)
delta = nyd - halloween
print(delta) #62 days, 0:00:00


#instantiate timedelta directly
#delta2 = datetime.timedelta(5, 7, 0, 0, 11) //same as below
delta2 = datetime.timedelta(days = 5, minutes = 11, seconds = 7)

print(delta2) #5 days, 0:11:07
```

https://docs.python.org/3.11/library/datetime.html#module-datetime

Hacking time

We can pause a program until a specific time or date:

```
import datetime
import time

final_exam = datetime.datetime(2026, 4, 17, 9)
while datetime.datetime.now() < final_exam:
    time.sleep(1)
```

datetime_example.py

https://docs.python.org/3.11/library/datetime.html#module-datetime

CLASSES AND OBJECTS

LET’S CREATE A CLASS!

Methods vs Functions

Functionally speaking, they are the same. They both ***perform an action/behavior*** and should be ***atomic***.

Methods act on an object. Changing its state and working with its attributes

- First parameter is ***'self'*** . This is the object it is defined on and is passed automatically
- Accessed through an object

```
def my_method(self, param1, param2):
  #execute method code
```

Functions are not object dependent

- Can be called without the assistance of objects
- Doesn't accept self as a parameter

```
def my_function(param1, param2):
  #execute function code
```

Methods vs Functions

Functionally speaking, they are the same. They both ***perform an action/behavior*** and should be ***atomic***.

Methods act on an object

- Change object's state and working with its attributes
- First parameter is '***self***' . This is the object it is defined on and is passed automatically
- Accessed through an object

```
class MyObject:
    def my_method(self, param1, param2):
        #execute method code

my_object = MyObject()
my_object.my_method(1,2)
```

Functions are not object dependent

- Can be called without the assistance of objects
- Doesn't accept self as a parameter

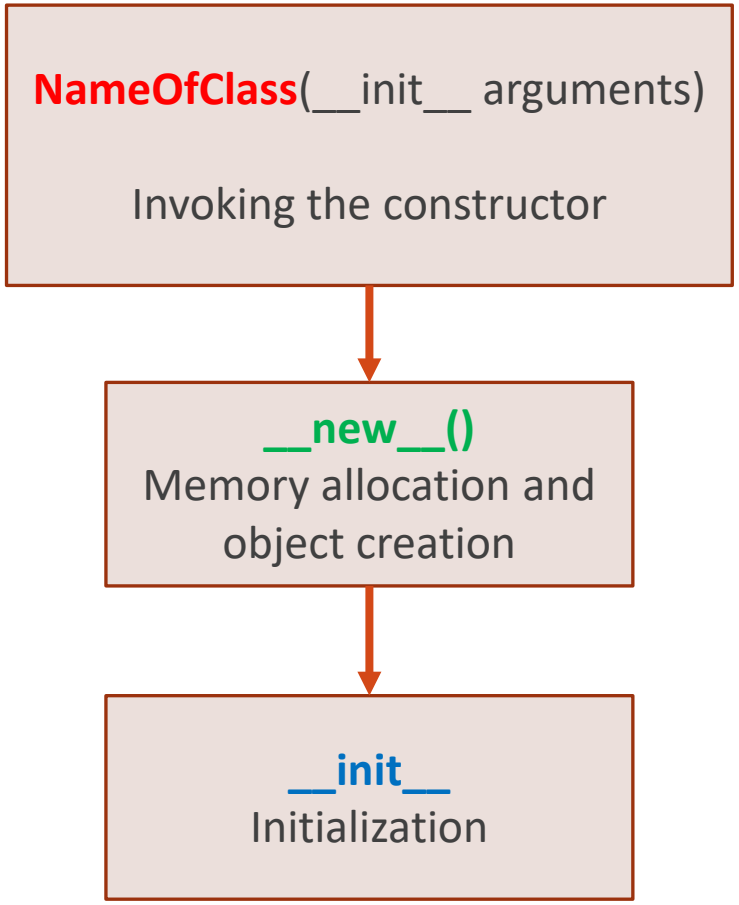
```
def my_function(param1, param2):
        #execute function code

my_function(1,2)
```

Instantiation

Suppose we have a class called Dice:

- Instantiate Dice by typing `Dice()`
- This calls a special method called `__new__`(we never tamper with __new__ and we never call it directly)
- `__new__` allocates memory and invokes the object initializer
- `Dice.__init__( )` is the **initializer**



Special method #1: __init__(self)

__init__(self) method

a) Special built-in method called the ***initializer***

b) First method inside the class definition

c) Executed once when a new object is instantiated

d) Note the 2 leading underscores and the 2 trailing underscores

e) Accepts one or more parameters:

   a) **self** – every method call associated with a class automatically passes self, which is a reference to the instance itself.

   b) data for the object's state

f) Note that the variables defined in the __init__ method are prefaced with self

Instance Variables?

In Python we often call them ***data attributes***!

We don't define them at the top of the class, that's a Java approach!

```java
//Java instance variables

class Cat {
    protected int _name = “”
    protected int _age = 0
    protected String _food_prefs = “”

    public Cat(String name, int age, String favorite_food)
    {
        _name = name
        _age = age
        _food_prefs = favorite_food
    }
}
```

```python
#Python data attributes

class Cat:
    def __init__(self, name, age, favorite_food):
        self._name = name
        self._age = age
        self._food_prefs = favorite_food
```

Instance Variables?

Python:

- Declare our variable and assign it a value inside the __init__(self) method
- Any variable prefaced with **self** in the __init__(**self**) method is available to every method in the class
- If it is **not prefaced with self**, it is just a *local variable*.

```
class Cat:
    def __init__(self, name, age, favorite_food):
        self._name = name
        self._age = age
        self._food_prefs = favorite_food
        another_var = 10 #this is a local variable
```

Out first class!

```
class Cat: #notice the class keyword
    def __init__(self, name, age, favorite_food):
        self._name = name
        self._age = age
        self._food_prefs = favorite_food


tobias = Cat("Tobias", 3, ["Sushi", "Raw fish"]) #instantiate cat
```

Notice the **self** parameter. All methods in a class need a self parameter, but it's not explicitly passed as a parameter when instantiating/calling methods

We didn't define **_name**, **_age**, **_food_prefs** class variables above the **__init__**

Declaring them inside the **__init__** is our way of declaring the instance variables

What about visibility?

There are no access or visibility modifiers in Python

Everything in scope is always visible and transparent

- Private
- Package default
- Protected
- Everything is public always and forever.

And that works?

Yes!

Access modifiers are not Pythonic. Python provides unenforced guidelines and best practices and trusts you not to misuse this.

Python developers ***trust*** other programmers to stick to the rules and use good OOP approaches.

There Is A Standard!

Developers still need to communicate that it is not safe to access certain attributes in a class/object.

Single Underscore

```
class Cat:
    def __init__(self, age):
        self._age = age
```

A single underscore communicates to other developers that they should ideally not be touching this variable and modifying it is not recommended.

Conceptually similar to "**protected**" instance variables

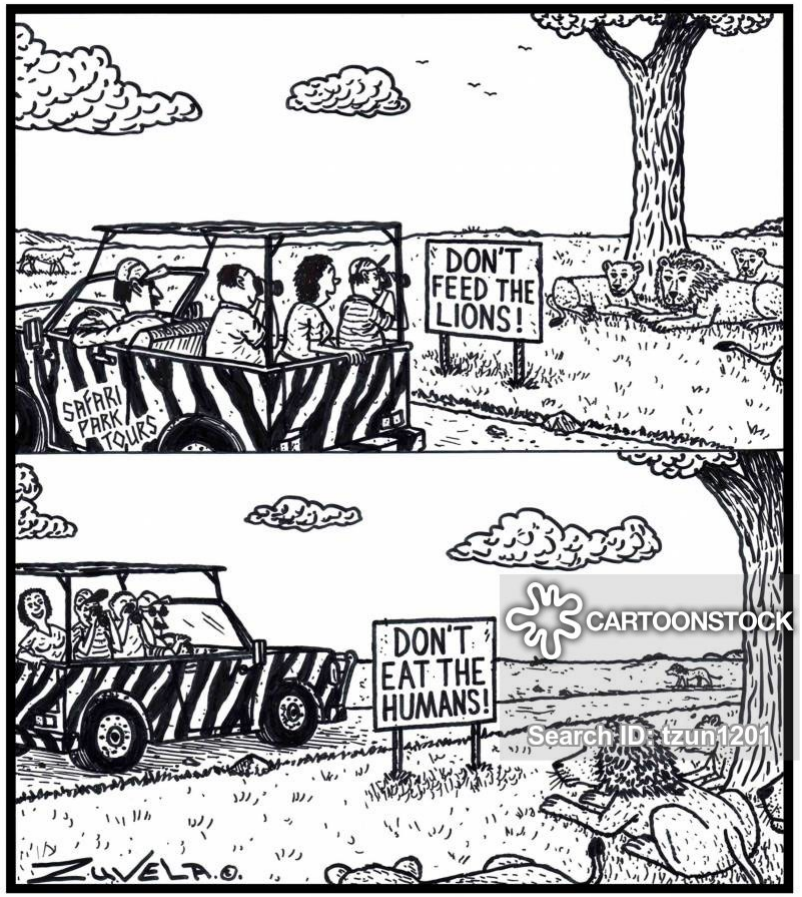
Double Underscores and Name Mangling

A variable name prefaced with double underscores (Dunder) is a way of saying “STAY AWAY”. Conceptually similar to “**private**” instance variables

Python makes it difficult to access this variable through **Name Mangling** , that means it’s only accessible by prefacing it with the **class name** and some more **underscores**.

Since it’s a bother to access the variables, developers are discouraged from detecting them and going near them.

```
class Cat:
    def __init__(self, age):
        self.__age = age
…
my_cat = Cat(10)
my_cat.age = 5 #doesn’t change cat __age. Creates new age variable
print(my_cat.__age) # ERROR
my_cat._Cat__age = 5 # Okay, but not good form
```



Special method #2: __repr__(self)

__repr__(self)

Is the same as Java's toString

Should contain unambiguous information about the current state of the object

Is implicitly called when we pass an object to the print( ) function

This is really for developers' eyes

Think of the information that should be in a log file

Method returns a string

"name: Tobias age: 3 food_prefs: ['Sushi', 'Raw fish'], num_lives: 9"

* Remember to say dunder for double under

Special method #3: __str__(self)

__str__(self)

Is also the same as Java's toString

Should be the same as the repr, except perhaps a little prettier (not a single line)

Not required

If not provided, __repr__(self) will always be used

Method returns a string

"My name is Tobias and I'm 3 years old.

love eating ['Sushi', 'Raw fish']

I have 9 lives remaining"

* Remember to say dunder for double under

Accessors & Mutators

Accessors: Methods that return the value of an attribute. *Think Java getter*.
Eg: `def get_player_health(self):`

Mutators: Methods that set the value of a data attribute. *Think Java setter*.
Eg: `def set_player_health(self, health_value):`

self parameter is the:

- *Only parameter* for all accessors
- *First parameter* for all mutators

Require docstring comments

Must not generate side effects

- Accessors must not modify or print or send messages
- Mutators may only modify a single value, and only if it remains logical

Properties vs Accessors / Mutators

A property is a way to encapsulate the getter (Accessor) and setter (Mutator) of an attribute.

Anyone interfacing the code with your library can still access the data attribute through a variable name.

The variable name automatically invokes the Accessor and Mutator.

Eg:

```
# set inside class definition
temperature = property(get_temperature, set_temperature)

..

..

my_engine.temperature = 70  # this would invoke set_temperature(70)
```

property_example.py

Static (Class) Variables

Java instances share static variables

Are there statics in Python? Yes!

Class variables are ***shared by all instances of a class***

A class variable:

1. is declared inside the class but not inside a method
2. is accessible directly (remember no access variables!)
3. can be accessed using ClassName.class_variable

```
class Cat:

    lives = 9 #class variable

print(Cat.lives)
```

Class Methods

Class Methods are like Class Variables.

- They belong to a class rather than the object.
- They take *cls* as their first parameter instead of self
- ***cls*** stands for the class itself.
- Add **`@classmethod`** before method
- *Can modify and access class variables but not instance variables*

While not quite like static functions, class methods can be accessed through the class name

```
@classmethod
def my_class_method(cls, arguments)
```

Class Methods

For example, say we had a temperature class that stored temperature as farenheit. We could write a class method that allows us to create objects using celcius

```
class Temperature:
        def __init__(self, tempFarenheit):
            self.temperature = tempFarenheit

        @classmethod
        def from_celcius(cls, tempCelcius):
            tempFarenheit = (tempCelcius * 9/5) + 32
            return cls(tempFarenheit)

farenheit_object = Temperature.from_celcius(50)
```

Static Methods

The **@staticmethod** function decorator allows static functions to be defined.

- Static functions belong to and can be accessed by the class
- Static functions do not require *cls* as its first parameter
- In fact, static functions can't access the class variables/object at all. They behave as independent functions within the scope of a class.

```
@staticmethod
def convert_to_fahrenheit(tempCelcius):
    farenheit = (tempCelcius * 9/5) + 32
    return farenheit


print(Temperature.convert_to_fahrenheit(100))
```

Static Methods vs Class Methods

Hold on, **Static Methods** and **Class methods** seem pretty similar, what's the difference?

Similarities:

- Both are class level methods that are called on the **class**, **NOT** the instance
- No need to instantiate an instance of the class before calling the method

```
class Temperature:
        def __init__(self, tempFarenheit):
            self.temperature = tempFarenheit

        @classmethod
        def from_celcius(cls, tempCelcius):
            # class method code

       @staticmethod
        def convert_to_fahrenheit(tempCelcius):
            # static method code
```

```
# code is called on the class directly, not an instance
temp = Temperature.from_celcius(50) #call class method
Temperature.convert_to_fahrenheit(100) #call static method
```

Static Methods vs Class Methods

Hold on, Static Methods and Class methods seem pretty similar, what's the difference?

Differences:

- Class Methods – perform operations **WITH** access to static class
- @classmethod decorator: use **cls** as first parameter. **cls** parameter automatically passed in.

```
@classmethod
def from_celcius(cls, tempCelcius):
    tempFarenheit = (tempCelcius * 9/5) + 32
    return cls(tempFarenheit)
```

- Static Methods – perform operations **WITHOUT** access to static class.
- @staticmethod decorator: do NOT use **cls** as first parameter.

```
@staticmethod
def convert_to_fahrenheit(tempCelcius):

    celcius = (tempCelcius * 9/5) + 32
    return celcius
```

All the concepts together!

Let's check out cat.py

- Class declaration
- Instance/class variables
- Initializer (self)
- Instantiation
- Methods
- Static, class methods (cls)
- _ ('protected' variables)
- __ (dunder)

Inheritance, Interfaces Protocols, Abstract classes

COMP 3522 OBJECT ORIENTED PROGRAMMING 2

PART 2

Agenda

1. Inheritance
2. Interface
3. Duck Typing
4. Protocols
5. Formal protocols and ABCs

INHERITANCE

Inheritance in Python

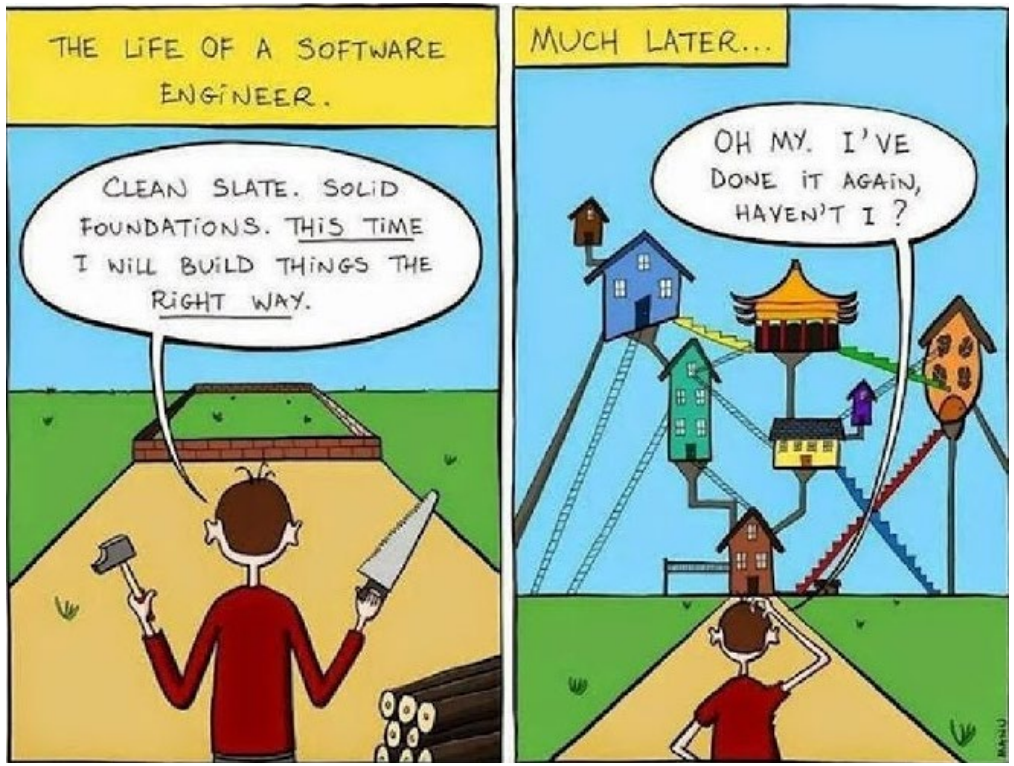
So easy

Almost the same approach as Java:

- Push common attributes as far up the inheritance hierarchy as possible
- Represent an is-a relationship
- Subclass (child class) is a specialized version of the superclass (parent class)

With one big difference:

- Multiple inheritance (oh the horror!)



Basic inheritance in Python

We specify the superclass as a parameter to the class definition:

```
class Subclass(Superclass):
    # class definition here
```

Basic inheritance in Python

Classes in Python implicitly extend the **object** class (just like Java)

```
class MySubClass:
    pass

#The above and below class declarations are equivalent

class MySubClass(object):
    pass
```

What can we extend

Almost anything!

Even the basic data types

1. string
2. int
3. float
4. List
5. All of them are classes

Overriding and super( )

We can extend behavior by adding new methods

We can also override existing methods like __init__( )

We can access the original using the super( ) function

super( ) returns the object as an instance of the parent class

This lets us call the parent method directly

We can call super( ) from inside any method

You should be thinking of order of initialization at this point…

Order of initialization

Remember in COMP 2522 we were very interested in the order of initialization

Suppose we have a simple inheritance hierarchy

In which order are the __init__ methods called? What's the output?

```
class Animal:
    def __init__(self):
        print('Animal')

class Dog(Animal):
    def __init__(self):
        print(‘Dog')

a = Dog()
```

```
Output: Dog
```

Order of initialization

In which order are static variables initialized? What's the output?

```
class Animal:
    print('A')
    animal_static_var = 5
    print('B')
    def __init__(self):
        print('Animal')

class Dog(Animal):
    print('C')
    dog_static_var = 10
    print('D')
    def __init__(self):
        print('Dog')

a = Dog()
```

Output:

A

B

C

D

Dog

Order of initialization

What about calls to super( )? When do they take place? What's the output?

```
class Animal:
    def __init__(self):
        print('Animal')

class Dog(Animal):
    def __init__(self):
        super().__init__()
        print('Dog')

a = Dog()
```

Output:
Animal
Dog

Order of initialization

What about calls to super( )? When do they take place? What's the output?

```
class Animal:
    def __init__(self):
        print('Animal')

class Dog(Animal):
    def __init__(self):
        print('Dog')
        super().__init__()

a = Dog()
```

Output:

Dog

Animal

Order of initialization

Is a call to super( ) implicit? Or do we have to make it **explicitly** in Python?

Must have **explicit** super()

Let’s see this in action.

Enemy

enemy_inheritance.py

INTERFACES

The Java interface

Recall the Java interface is a set of methods and constants

It defines how to interact with a “kind” of thing

The concept of an interface exists in Python, too

There are two kinds:

1. **Informal interfaces** (convention **NOT enforced** by the language.)
   a) Duck typing
   b) Protocols
2. **Formal interfaces** (contract **enforced** by the language)
   a) Abstract base classes

Informal interfaces:
DUCK TYPING

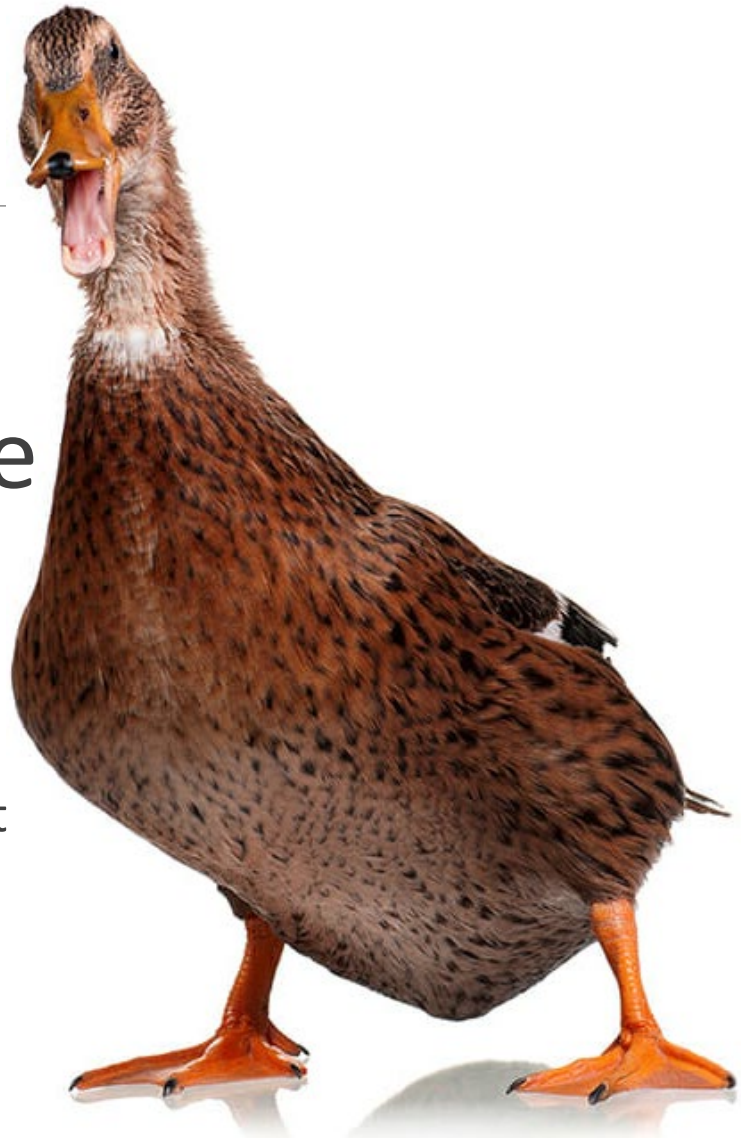
Duck typing

We say that Python uses **DUCK TYPING:**

"If a bird walks, swims, and quacks like a duck, then call it a duck”

For example, if an object can be concatenated, indexed, and converted to ASCII, that is, if it does everything a string can do, then let’s treat the object like a string.

Duck typing is when an object's suitability for typing takes place at runtime



Python uses dynamic typing

A Python programmer can use any type of object as an argument to a function

Consider a function add(a, b) that adds the two parameters

```
def add(a, b):
    return a + b
```

We can invoke this function using ints or using strings!

```
print(add(1, 1)) #2
print(add('nanoo', 'nanoo’)) #nanoonanoo
```

More dynamic typing

Python uses **dynamic typing** to determine the type of objects during runtime

For example, the consecutive statements

num = 5 # first assign num to an integer type

num = '7' # and then a string type

The type of num can change, depending on the value it references

<u>The interpreter is responsible for checking that all operations are valid as the program executes</u>

If the function call add(5, '100') is evaluated, the interpreter generates an error when we try to add the string to an integer.

Static typing

In contrast to dynamic typing, many other languages like C, C++, and Java (of course) use**static typing**

Static typing requires the programmer to **define the type** of **every variable** and **every function parameter** in a program's source code

For example: `int answerToEverything = 42;` to declare an int variable

When the source code is compiled, the compiler attempts to detectnon type-safe operations, and halts the compilation process if such an operation is found.

Which is better?

Dynamic typing typically gives the programmer more flexibility than static typing

Dynamic typing can potentially introduce more bugs, because there is no compilation process that checks types

We say that Python is strongly typed

The interpreter keeps track of object types

We are not allowed to do anything that is incompatible with the type of data we are working with

We cannot perform operations inappropriate for the type of the object

Q: How can we prove this?

A: Try to add a number to a string!

(We can do this in weakly typed languages like JavaScript, but not Python)

What does this mean?

Suppose you have a function that accepts an object

It expects the object to have a method called **list**

We don't have to specify what type the object is

The code will compile and execute as long as the object, whatever it is, has a method called **list**

This is great!

This is **runtime polymorphism**! This is fabulous advanced runtime polymorphism!

"We don't care what kind of object you are, as long as you have a list( )!"

polymorphism.py, polymorphism_2.py

Informal interfaces:
PROTOCOLS

The Python protocol

Protocols are collections of methods that Python developers and libraries expect certain “kinds” of things to have

This is analogous to **interfaces** in the Java Collections Framework

There are some very helpful protocols in Python

In order to implement a protocol, a type must support certain operations

The Sized protocol

Implemented by having a method called `__len__(self)`

Invoked when an instance is passed to the built-in function `len( )`

This is how an object reports the correct value to the `len( )` function

We may assume Python's built-in len( ) function works something like this:

```
def len(obj):
    return obj.__len__()
```

`__len__(self)` returns the length of the object, an integer >= 0

Implemented by str, list, range, tuple, bytes, set,dict

The Container protocol

Provides the method **`__contains__`**`(self, item)`

When the membership operator **in** or **not in** is invoked on an object, **this is the method invoked**

This is how a class implements the membership test operator

`__contains__``(self, item)` must return true if the item is **in** self, else false

Implemented by str, list, range, tuple, bytes, set,dict

The Sequence protocol

Permits the following:

1. Retrieving an item by index, i.e., item = seq[index]
2. Finding an item by value, i.e., index = seq.index(item)
3. Counting items, i.e., num = seq.count(item)
4. Producing a reverse iterator, i.e., r = reversed(seq)

Must implement

1. count(item)
2. index(item)
3. __len__( )
4. __getitem__(self, key)
5. __contains__(self, item)
6. __reversed__(self)

Implemented by str, list, range, tuple, bytes

And many more!

For example:

- An **Iterator** must provide:
  - **__next__**(self)
  - **__iter__**(self)
- A **Collection** must provide
  - **__len__(**self),
  - **__iter__**(self), and
  - **__contains__**(self, item)

Check out https://docs.python.org/3/library/typing.html#classes-functions-and-decorators

Aside: so many dunders (__)

There are many methods in Python that begin and end with dunder

Sometimes we call these **magic methods**

These are usually the methods we need to implement in order to conform to protocols

We can also overload operators with these

FORMAL INTERFACE: Abstract Base Classes (ABCs)

Formal interface: Abstract Base Class (ABC)

In Java, **Interfaces** and **Abstract** classes are different. **Interfaces** only define method signatures, while **abstract classes** can define method signatures and attributes.

Abstract Base Classes are Python's answer to Interfaces and Abstract Classes!

There are some formal interfaces

Python provides **A**bstract **B**ase **C**lasses (**ABC**s)

The ABC complements duck-typing

ABCs define a set of **methods** and **properties** that a class must implement in order to be considered a duck-type instance of that class

Python provides built-in ABCs for data structures, numbers, streams, etc.

We can also create our own ABCs with the abc module

https://docs.python.org/3/library/abc.html#module-abc

Example

```
import abc

class Bird(abc.ABC):

    @abc.abstractmethod
    def fly(self):
        pass

    @abc.abstractmethod
    def lay_egg(self):
        pass
```

Import from **abc module** to use Abstract Base Classes

Abstract methods meant to be overridden in subclasses are **decorated** with **@abc.abstractmethod**

The **Bird** class acts as a “blueprint” for future subclasses. It can not be instantiated

Abstract methods tagged and mandatory

```
class Parrot(Bird):

    def fly(self):
        print("Flying")

    def lay_egg(self):
        print(“Laying egg")

p = Parrot()
print(isinstance(p, Bird)) #True
```

An **implementing class** is a subclass of the ABC

If an abstract method is not implemented by a subclass, Python will generate a TypeError: Can't instantiate abstract class X with abstract methods Y

abc_example.py, abstractbaseclass.py

That’s it for Week 2!

Next Week:

UML

SOLID Design Principles

Dependencies & Coupling